

Prepared from the current BIGTIME POS repository implementation as of April 13, 2026.

BIGTIME POS System Description

Prepared on: April 13, 2026

System name: BIGTIME POS

Document purpose: Formal description of the full BIGTIME POS platform, covering the dashboard, central API, database services, and Android/Windows cashier terminal application.

1. Executive Summary

BIGTIME POS is an integrated point-of-sale and back-office management system designed for retail and cashiering operations. The platform is composed of:

- a web-based back-office dashboard for administrators and authorized office users
- a central server API that handles authentication, business rules, transaction processing, reporting, and compliance services
- a POS terminal application built in Flutter for Android and Windows cashier terminals
- central data services for persistent records, synchronization, and operational messaging

The system supports day-to-day selling, cashier shift management, item and inventory administration, payment processing, receipt issuance, report generation, audit visibility, and offline transaction continuity for the POS terminal.

2. System Scope

The BIGTIME POS platform is intended to manage retail sales operations from back-office setup to front-counter transaction completion. It covers:

- dashboard-based administration and maintenance of master data
- cashier login and shift operations
- selling, cart, payment, and receipt generation
- transaction synchronization between terminal and server
- inventory and catalog management
- employee and POS user administration
- BIR-related and EIS-related reporting functions
- customer-facing display support

The current implementation is organized as a monorepo with these major applications:

- `apps/dashboard-react`

- `apps/api-nestjs`
- `apps/pos-flutter`
- `packages/shared-types`

3. System Architecture

3.1 High-Level Architecture

BIGTIME POS follows a multi-application client-server architecture.

- The dashboard is a browser-based React application used by administrators and back-office personnel.
- The API is a NestJS server that exposes authenticated endpoints for POS operations, dashboard management, sync, reports, and compliance-related functions.
- The POS terminal is a Flutter application used by cashiers on Android tablets or Windows terminals.
- PostgreSQL serves as the central database for transactional and master records.
- Redis supports queueing and related runtime infrastructure.
- The POS terminal maintains a local SQLite database using Drift for offline continuity and queued sync records.

3.2 Central Components

A. Dashboard Application

The dashboard provides administrative access to operational data and system management functions. Based on the current implementation, the dashboard includes routes and modules for:

- Dashboard overview
- Receipts
- Catalog
- Listing maintenance:
 - items
 - categories
 - category groups
 - discounts
 - taxes
 - payment methods
- Inventory:
 - warehouse
 - item stocks
 - item transfers

- CSV import
 - suppliers
 - purchase orders
- Reports:
 - sales
 - branch comparison
 - shift
 - cash balancing
 - reference number
 - expiration date
 - discount
 - pull-out
 - BIR taxes
 - BIR terminal report
 - BIR eSales / EIS
- Employees:
 - employees
 - work hours
 - time card
- Settings
- POS users
- Account

B. Central API

The API is the core business layer of the system. It currently contains modules for:

- account
- auth
- bir
- catalog
- customer-display
- eis
- employees
- inventory
- payments
- pos

- pricing
- reports
- sync

Additional infrastructure in the API includes:

- Prisma for database access
- Redis-backed queue infrastructure
- realtime gateway support
- global validation, throttling, and security middleware

C. POS Terminal Application

The POS terminal is the cashier-facing application. Based on the current app flow, the terminal progresses through these main stages:

- Login
- Open Shift
- Selling
- Payment
- Receipt

Major POS functional areas in the Flutter app include:

- authentication
- app flow control
- catalog access and selling workspace
- customer display
- shift management
- payment and receipt flow
- local queue and sync operations

4. Users and Access Levels

The platform supports multiple user types depending on operational responsibilities.

4.1 Cashier

The cashier uses the POS terminal application to:

- log in using employee code and PIN
- open a shift with an opening cash float
- add items to cart

- accept payment
- issue receipts
- reprint or email receipts where supported
- close the shift and perform cash reconciliation

4.2 Supervisor or Manager

Supervisory users may be involved in:

- variance approval during shift close
- operational review of transactions
- oversight of staff and terminal activity
- review of compliance-related outputs

4.3 Administrator / Back-Office User

The dashboard is intended for administrative or authorized users who manage:

- catalog and listing data
- inventory and stock operations
- employee and POS user records
- reports and monitoring
- BIR and EIS-related views
- customer display settings and operational configuration

5. Functional Description by Layer

5.1 Dashboard Layer

The dashboard is the management interface of BIGTIME POS. It is used to maintain centralized data and review operational outputs generated by the cashier terminals and API.

Its functional responsibilities include:

- authentication of authorized back-office users
- viewing operational summaries and receipts
- maintenance of item catalog, categories, taxes, discounts, and payment methods
- inventory monitoring and stock workflows
- employee and POS user administration
- report access for sales and statutory review
- BIR and EIS monitoring functions

- settings and customer display configuration

The dashboard consumes server APIs and reflects centrally stored data. It does not operate as a standalone terminal and is dependent on the API and central database.

5.2 API and Business Logic Layer

The API is responsible for enforcing business rules and serving as the integration point for both the dashboard and cashier terminal.

Its main functions include:

- user authentication and token handling
- role-based protection of protected resources
- transaction posting and receipt numbering
- payment recording
- synchronization intake for offline POS activity
- report generation and compliance-related endpoints
- catalog, pricing, and inventory services
- employee and user management
- audit visibility and receipt event logging
- customer display and realtime support

The API serves as the authoritative source of official transaction data whenever the terminal is online.

5.3 POS Terminal Layer

The POS terminal is optimized for cashier operations and is designed to continue functioning even when connectivity is temporarily unavailable.

The POS application performs the following:

- authenticates the cashier
- stores shift opening information locally
- reads catalog data from local cache
- builds carts and computes amounts on the device
- processes payment input
- issues receipts
- stores transaction and event data in the local queue when offline
- synchronizes queued records to the server when connectivity is restored

The POS app also contains hardware integration hooks for:

- receipt printing

- cash drawer pulse
- customer-facing display control

6. End-to-End Operational Flow

6.1 Master Data Preparation

Before daily selling begins, master records such as items, categories, taxes, discounts, payment methods, and inventory-related data are maintained through the dashboard. These records are stored in the central database and are later consumed by the POS terminal through server synchronization.

6.2 Cashier Login

The cashier opens the POS application and enters an employee code and PIN. The terminal calls the authentication endpoint of the API. If credentials and permissions are valid, the terminal receives authenticated session context and proceeds to shift opening.

6.3 Shift Opening

The cashier enters the opening cash float. The terminal records the shift locally and queues the shift event for synchronization. When online, the terminal attempts to upload the shift information to the server immediately.

6.4 Selling and Cart Build

The selling screen uses locally cached catalog data for responsive operation. The cashier may search by item name, SKU, or barcode, and may adjust quantities before charging the cart. Pricing, totals, and VAT computations are performed in the app based on the loaded cart state and current item data.

6.5 Payment

The payment screen supports payment method selection and amount capture. Based on the current implementation, payment methods include:

- Cash
- Card
- GCash
- Maya
- Split

For cash payments, the tendered amount is entered and change is computed. For non-cash methods, the terminal can capture reference-related information as required.

6.6 Receipt Issuance

If the terminal is online:

- the POS app sends the transaction to the API
- the server records the transaction centrally
- the server returns the official transaction identifier, OR number, and reference number

If the terminal is offline or the live server path cannot be completed:

- the POS app generates a local provisional transaction record
- the transaction is saved to the local sync queue
- once the terminal reconnects, the queued record is uploaded
- the server reconciles the local record and returns the final identifiers and receipt information

This design allows selling continuity while preserving later reconciliation to server-issued records.

6.7 Receipt Actions and Audit Visibility

After sale completion, the terminal can perform receipt-related actions such as print, reprint, and other receipt events supported by the platform. These events are queued and synchronized so that the backend has a visible audit trail of receipt-related activity.

6.8 Shift Close and Reconciliation

At end of shift, the terminal computes expected cash based on opening cash and sales activity. The cashier enters the blind count. If configured thresholds are exceeded, supervisor or manager intervention may be required. The shift-close data is recorded locally and synchronized to the server when possible.

6.9 Dashboard Monitoring and Reporting

Once data is in the central system, the dashboard provides access to receipts, sales data, inventory views, employee-related views, and reports. The dashboard becomes the back-office reporting and management layer for transactions generated by the POS terminals.

7. Data Storage and Records

7.1 Central Database

The central database uses PostgreSQL. Based on the Prisma schema, the central data model includes records such as:

- branches
- users
- terminals
- shifts

- category groups
- categories
- items
- item variants
- taxes
- discounts
- transactions
- transaction items
- payments
- inventory and stock movement-related records
- employees and related work records

The central database acts as the primary permanent record for official operational data.

7.2 Local POS Database

The POS application uses a local SQLite database through Drift. The local database currently contains tables for:

- `catalog_cache_items`
- `sync_queue_entries`
- `shift_sessions`

This local database supports:

- fast local item lookup
- terminal continuity during connectivity interruptions
- deferred upload of pending records
- temporary retention of shift and sync state

7.3 Queue and Runtime Infrastructure

Redis is used as part of the server runtime infrastructure. The current deployment uses Redis with append-only persistence enabled. Queue and realtime components support background processing and operational messaging inside the system.

8. Security and Control Features

Based on the current implementation and supporting documentation, BIGTIME POS includes the following control features:

- JWT-based authentication for protected API routes

- refresh-token handling for authenticated sessions
- role-based access control
- route protection for dashboard access
- request validation using DTO validation and whitelist enforcement
- throttling and abuse protection
- security middleware such as Helmet
- hashed PIN handling for managed user credentials
- traceability for transactions, receipts, and sync-related records
- idempotent sync handling and retry controls for queued data

These controls are intended to reduce unauthorized access, malformed requests, duplicate sync uploads, and untracked transaction actions.

9. Network and Offline Behavior

BIGTIME POS is designed for mixed online and offline operation.

9.1 Online Operation

During normal connected operation:

- the dashboard reads and updates central records through the API
- the POS terminal authenticates against the API
- transactions are posted directly to the server
- official references and OR values are returned immediately
- reports and dashboards reflect centrally stored records

9.2 Offline POS Operation

If the POS terminal loses connectivity:

- local cached catalog data continues to support selling
- transactions and shift events are stored in the local queue
- receipt-related events remain locally recordable
- queued records are uploaded once connectivity is restored

This enables continuity at the cashier terminal while preserving a later synchronization path to the central server.

10. Backup, Recovery, and Continuity

10.1 Central Backup Scope

For operational continuity, the central backup scope should include:

- PostgreSQL data
- Redis runtime persistence where operationally required
- application configuration and environment files used for deployment
- generated reports or exported compliance files retained outside the database, if any

In the current local deployment, PostgreSQL and Redis are persisted through Docker volumes:

- `bigtime_pos_postgres_data`
- `bigtime_pos_redis_data`

10.2 POS Terminal Backup Scope

For terminal continuity, the local POS database file should be included in device-level backup or controlled retrieval procedures. The existing documentation identifies the local SQLite database as:

- `apex_pos.sqlite` in the app documents directory

This local file contains cached catalog data, queued sync entries, and local shift session data needed for terminal continuity.

10.3 Recovery Approach

In a recovery scenario, the recommended restoration sequence is:

1. restore the central database and required runtime services
2. restore deployment configuration and start the API
3. restore and start the dashboard application
4. relaunch POS terminals
5. allow terminals to reconnect and upload pending local queue entries

Because the POS app maintains local queue records, pending terminal transactions can be synchronized after the server environment becomes available again, subject to the integrity of the local terminal database.

11. Deployment and Runtime Environment

The current workspace supports these main runtime components:

- API server: `apps/api-nestjs`
- Dashboard web app: `apps/dashboard-react`
- POS terminal app: `apps/pos-flutter`
- Central PostgreSQL database

- Redis service

In the current local environment:

- the API default base path is `http://localhost:3000/api`
- the dashboard runs as a browser-based React application
- PostgreSQL is exposed on local port `54329`
- Redis is exposed on local port `6380`

The POS terminal can run on:

- Android tablets
- Windows terminal devices

12. Summary

BIGTIME POS is a full-stack retail transaction system composed of a web dashboard, a central API, and a cashier terminal application for Android and Windows. The dashboard manages setup, administration, and reporting. The API handles authentication, transaction logic, synchronization, and centralized records. The POS terminal supports cashier workflows, receipt issuance, and offline continuity through local storage and sync queueing.

Taken together, these components provide one integrated system from back-office management down to front-counter transaction execution and receipt generation.